

Roadmap V3

Rank-Adaptive and Sparsity-Aware Dynamic Quadtree Matrix Multiplication over Containerized Microservices

Course Option: CSC 4770 Option 1 — Graduate / 5770 Requirements

Project Type: Parallel and distributed matrix multiplication using Docker microservices, TCP sockets, dynamic task scheduling, and structure-aware communication reduction.

1. Project Objective

The system will multiply two user-supplied matrices by decomposing them into smaller matrix blocks, distributing block-multiplication tasks across multiple Dockerized worker microservices, and reconstructing the final result in a dedicated aggregation service.

The baseline implementation will satisfy the course requirements:

- separate Docker containers for microservice roles,
- TCP socket communication,
- divide-and-conquer / blocked matrix multiplication,
- parallel worker execution,
- result aggregation,
- horizontal worker scaling,
- dynamic workload distribution,
- error handling and clear failure reporting.

The graduate-level extension evaluates whether **numerical structure within matrix blocks** can be exploited to reduce communication and computation costs in a containerized environment.

The two adaptive mechanisms are:

1. **Approximate sparsity pruning** using Frobenius-norm product bounds.
2. **Low-rank factored transmission** using truncated matrix factorization.

The project is framed as a systems experiment rather than as an attempt to outperform mature HPC libraries.

2. Research Question and Hypotheses

2.1 Primary Research Question

To what extent can sparsity-aware pruning and rank-adaptive block compression reduce communication volume and total execution time in TCP-connected containerized matrix multiplication, and what numerical error is introduced by those optimizations?

2.2 Hypotheses

For structured matrices containing sufficient sparsity or low numerical rank,

$$T_{\text{adaptive}} < T_{\text{static}}$$

primarily because

$$D_{\text{adaptive}} < D_{\text{static}},$$

where T denotes execution time and D denotes network data volume.

For dense, unstructured, approximately full-rank matrices, adaptive preprocessing may add overhead:

$$T_{\text{adaptive}} \geq T_{\text{static}}.$$

This outcome is not considered a failure. It identifies the matrix regimes in which adaptive preprocessing is beneficial.

3. Mathematical Notation

Let:

- $A, B \in \mathbb{R}^{N \times N}$: input matrices.
- $C = AB$: exact matrix product.
- N : original matrix dimension.
- N' : padded matrix dimension.
- b_{leaf} : leaf-level block dimension.
- d : quadtree depth.
- $b_d = N' / 2^d$: block dimension at recursion depth d .
- $q = N' / b_{\text{leaf}}$: number of leaf blocks along one matrix dimension.
- $A_{ik}, B_{kj} \in \mathbb{R}^{b \times b}$: leaf-level matrix blocks.
- $T_{ikj} = A_{ik} B_{kj}$: partial product.
- $C_{ij} = \sum_{k=1}^q T_{ikj}$: output block.
- P : number of active worker containers.
- $\epsilon_{\text{zero}} > 0$: approximate-pruning tolerance.
- $\epsilon_{\text{rank}} > 0$: relative low-rank reconstruction tolerance.
- r_{max} : maximum accepted numerical rank.
- α : fixed per-message communication latency term.
- β : per-byte transmission cost.
- γ : effective floating-point compute coefficient.
- $S_P = T_1 / T_P$: parallel speedup.
- $E_P = S_P / P$: parallel efficiency.

The Frobenius norm is

$$\|M\|_F = \sqrt{\sum_{r=1}^m \sum_{c=1}^n |M_{rc}|^2}.$$

4. Quadtree-Compatible Dimension Alignment

A standard multiple-of- b_{leaf} padding rule is not sufficient to guarantee a clean binary quadtree. The padded dimension must be reachable from b_{leaf} through repeated powers of two.

Define

$$d = \max \left(0, \left\lceil \log_2 \left(\frac{N}{b_{\text{leaf}}} \right) \right\rceil \right).$$

Then

$$N' = b_{\text{leaf}} 2^d$$

and therefore

$$N' \rightarrow \frac{N'}{2} \rightarrow \frac{N'}{4} \rightarrow \cdots \rightarrow b_{\text{leaf}}$$

is always valid.

The original matrices are embedded into zero-padded matrices:

$$\tilde{A}_{r,c} = \begin{cases} A_{r,c}, & 1 \leq r, c \leq N, \\ 0, & \text{otherwise,} \end{cases}$$

$$\tilde{B}_{r,c} = \begin{cases} B_{r,c}, & 1 \leq r, c \leq N, \\ 0, & \text{otherwise.} \end{cases}$$

After aggregation,

$$C = \tilde{C}[1 : N, 1 : N].$$

5. Divide-and-Conquer Matrix Decomposition

For a 2×2 quadrant decomposition,

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}.$$

The result is

$$C_{11} = A_{11}B_{11} + A_{12}B_{21},$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22},$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21},$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}.$$

At leaf level, with $q = N'/b_{\text{leaf}}$,

$$C_{ij} = \sum_{k=1}^q A_{ik} B_{kj}$$

and the total number of leaf block-multiplication tasks is

$$N_{\text{tasks}} = q^3.$$

6. Adaptive Sparsity Pruning

Before dispatching a block multiplication, the Ingress service computes

$$\|A_{ik}\|_F \quad \text{and} \quad \|B_{kj}\|_F.$$

By submultiplicativity of the Frobenius norm,

$$\|A_{ik} B_{kj}\|_F \leq \|A_{ik}\|_F \|B_{kj}\|_F.$$

If

$$\|A_{ik}\|_F \|B_{kj}\|_F \leq \epsilon_{\text{zero}},$$

the task is not transmitted.

The implementation substitutes

$$\hat{T}_{ikj} = 0.$$

This is **approximate numerical pruning**, not exact-zero elimination.

The local approximation error is bounded by

$$\|T_{ikj} - \hat{T}_{ikj}\|_F = \|T_{ikj}\|_F \leq \epsilon_{\text{zero}}.$$

The final result is denoted $\hat{C}$, and numerical accuracy is measured using

$$\text{Relative Error} = \frac{\|C - \hat{C}\|_F}{\|C\|_F}.$$

For experiments requiring exact arithmetic behavior, set

$$\epsilon_{\text{zero}} = 0$$

or disable pruning.

7. Rank-Adaptive Factored Transmission

7.1 Low-Rank Representation

For a candidate block A_{ik} , compute a truncated factorization

$$A_{ik} \approx U_A \Sigma_A V_A^T.$$

Define

$$X_A = U_A \Sigma_A, \quad Y_A = V_A,$$

so that

$$A_{ik} \approx X_A Y_A^T,$$

where

$$X_A, Y_A \in \mathbb{R}^{b \times r_A}.$$

A block is accepted as low-rank if

$$\frac{\|A_{ik} - X_A Y_A^T\|_F}{\|A_{ik}\|_F} \leq \epsilon_{\text{rank}}$$

and

$$r_A \leq r_{\max}.$$

The same procedure may be applied independently to B_{kj} .

7.2 Transmission Cost

A dense $b \times b$ float64 block requires

$$8b^2$$

bytes.

A factored block requires approximately

$$8br + 8br = 16br$$

bytes.

The payload compression ratio is therefore

$$R_{\text{payload}} = \frac{16br}{8b^2} = \frac{2r}{b}.$$

Compression is useful only when

$$r < \frac{b}{2},$$

and in practice a stricter configurable threshold should be used to ensure that factorization overhead is justified.

7.3 Compute-Aware Worker Paths

Case A: A is low-rank, B is dense

If

$$A \approx X_A Y_A^T,$$

then

$$AB \approx X_A (Y_A^T B).$$

This can reduce arithmetic cost from approximately

$$O(b^3)$$

to

$$O(b^2 r_A).$$

Case B: A and B are both low-rank

If

$$A \approx X_A Y_A^T, \quad B \approx X_B Y_B^T,$$

then

$$AB \approx X_A (Y_A^T X_B) Y_B^T.$$

This permits the worker to exploit factorization directly instead of reconstructing both dense operands before multiplication.

7.4 Cost-Control Rule

Low-rank analysis will be treated as an optimization layer, not as a prerequisite for correctness.

The implementation should:

1. skip factorization for blocks below a configurable minimum size,
2. enforce $r \leq r_{\max}$,
3. reject factorizations that do not meet ϵ_{rank} ,
4. reject factorizations whose predicted byte savings are insufficient,
5. measure factorization time separately from network and worker compute time.

A truncated or randomized method is preferred over a full exact SVD for large blocks.

8. TCP Communication Model

8.1 Persistent Connections

Workers should maintain persistent TCP connections to the scheduler/Ingress and result path whenever practical.

The system should **not** create a fresh TCP connection for every block multiplication.

This reduces:

- connection setup overhead,
- repeated handshakes,
- socket churn,
- latency noise during benchmarks.

8.2 Framed Binary Protocol

TCP is an undelimited byte stream. Every application-layer message therefore uses a fixed-size binary header followed by a length-delimited payload.

A receiver:

1. reads exactly the fixed header length,
2. validates protocol version and message type,
3. extracts payload length,
4. performs a bounded receive loop until the complete payload has been read,
5. validates dimensions and payload encoding,
6. dispatches the decoded message.

Payloads use contiguous binary `float64` buffers rather than JSON/XML numerical arrays.

8.3 Message Categories

The protocol must distinguish at least:

- `TASK_DENSE`
- `TASK_A_FACTORED`
- `TASK_B_FACTORED`
- `TASK_BOTH_FACTORED`
- `RESULT_RETURN`
- `HEARTBEAT`
- `ERROR`

A task must carry:

- task ID,
- output block coordinate (i, j) ,
- reduction index k ,
- matrix/block dimensions,
- payload encoding,
- payload byte length.

9. Corrected Communication Complexity

Let

$$q = \frac{N'}{b}.$$

There are

$$q^3$$

leaf block-multiplication tasks.

For the static dense baseline, each task sends two dense $b \times b$ float64 operands:

$$D_{\text{dispatch/task}} = 2(8b^2) = 16b^2.$$

Therefore,

$$D_{\text{dispatch}} = 16q^3b^2.$$

Since $q = N'/b$,

$$D_{\text{dispatch}} = 16 \frac{N'^3}{b}.$$

If every task returns one dense $b \times b$ partial product,

$$D_{\text{return/task}} = 8b^2$$

and

$$D_{\text{return}} = 8 \frac{N'^3}{b}.$$

Ignoring framing bytes,

$$D_{\text{dense,total}} = 24 \frac{N'^3}{b}.$$

This exposes the key block-size tradeoff:

- smaller b increases scheduling flexibility and available parallelism,
- smaller b also increases task count and total communication volume.

For a framed task protocol, message latency scales with message count. If each task requires one dispatch message and one result message,

$$N_{\text{messages}} \approx 2q^3.$$

A first-order communication model is

$$T_{\text{comm}} \approx \alpha N_{\text{messages}} + \beta D_{\text{effective}},$$

where $D_{\text{effective}}$ is the measured post-pruning, post-compression network payload.

10. Computation and Total-Time Model

Dense serial matrix multiplication requires

$$O(N^3)$$

work.

Under an idealized balanced P -worker model,

$$T_{\text{compute}} \approx \frac{\gamma N'^3}{P}.$$

If a fraction σ of block tasks is pruned,

$$T_{\text{compute,pruned}} \approx \frac{(1 - \sigma)\gamma N'^3}{P}.$$

A practical total-time model is

$$T_{\text{total}} = T_{\text{ingress}} + T_{\text{queue}} + T_{\text{comm}} + T_{\text{worker}} + T_{\text{aggregation}}.$$

For modeling purposes,

$$T_{\text{comm}} \approx \alpha N_{\text{messages}} + \beta D_{\text{effective}}.$$

The implementation should measure each major component independently rather than assuming all overhead is captured by a single asymptotic expression.

11. Dynamic Work Scheduling

Static worker assignment is not sufficient because adaptive tasks may have heterogeneous costs.

The scheduler therefore maintains a shared task queue

$$Q = \{T_1, T_2, \dots, T_n\}.$$

Workers obtain tasks dynamically when they become available.

Preferred policy:

Pull-based dynamic scheduling: an idle worker requests the next available task.

Advantages:

- reduces idle time,
- prevents one worker from receiving all expensive tasks,
- naturally adapts to sparse/pruned workloads,
- naturally adapts to dense vs. factored task types,
- satisfies the course requirement for intelligent workload distribution.

The scheduler records:

- queue wait time,
- worker assignment,
- task start/end time,
- retries,
- failure state.

12. Error Handling and Fault Behavior

The protocol and scheduler should explicitly handle:

- invalid protocol magic/version,
- unknown message type,
- malformed dimensions,
- invalid payload length,
- payload under-read,
- socket timeout,
- worker disconnect,
- task execution failure,
- duplicate task ID,
- result for unknown task,
- aggregation mismatch.

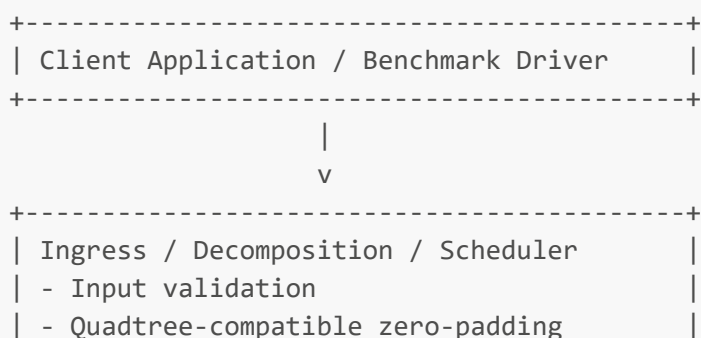
A failed task may be retried on another healthy worker up to a configurable retry limit.

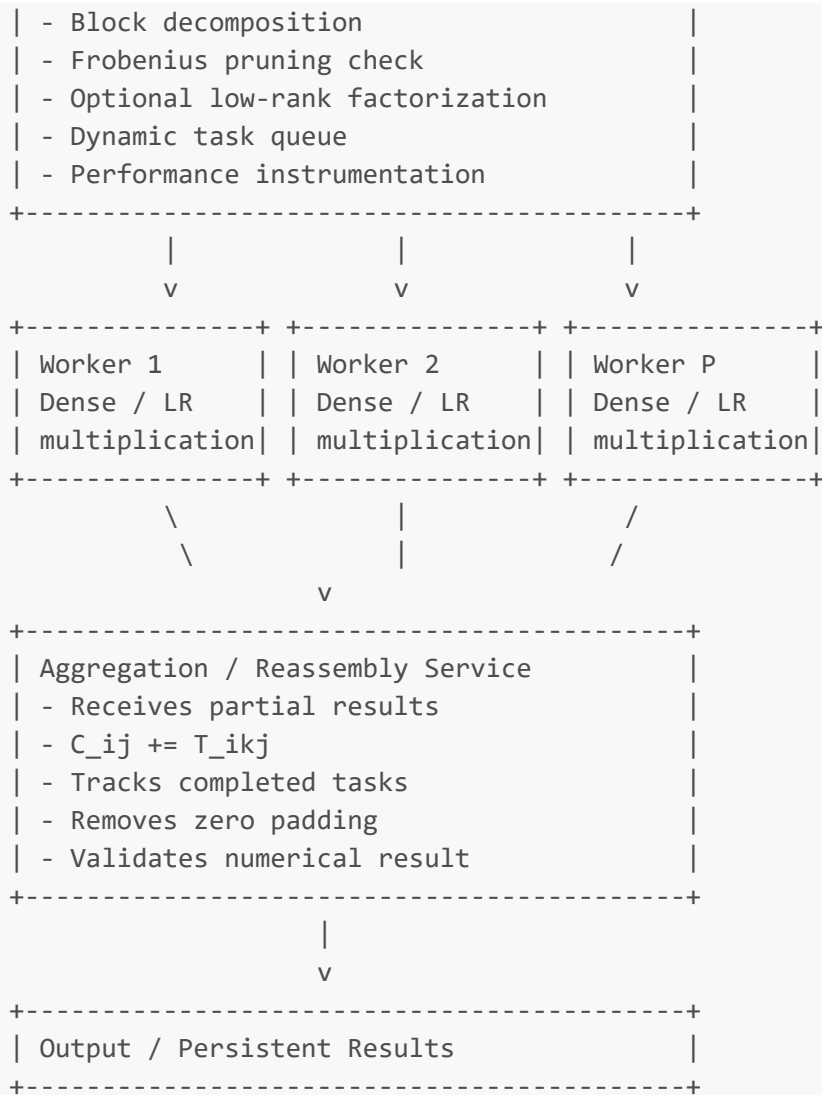
Errors must be reported with:

- task ID,
- worker ID,
- error category,
- concise diagnostic message.

The project does not require production-grade distributed consensus or orchestration.

13. Containerized Microservice Topology





14. Experimental Design

14.1 Systems to Compare

The experiment will use at least three implementations.

Baseline A — Serial

Single-process dense multiplication.

Purpose:

- establish correctness,
- establish T_1 ,
- provide a non-distributed timing baseline.

Baseline B — Static Docker Block Multiplication

Containerized TCP implementation with:

- dense blocks,
- no sparsity pruning,
- no low-rank compression.

Purpose:

- isolate the benefits and costs of parallelism and containerization.

System C — Adaptive Docker Block Multiplication

Same container architecture plus:

- approximate sparsity pruning,
- rank-adaptive transmission,
- dynamic task scheduling.

Purpose:

- isolate the contribution of the proposed adaptive mechanisms.

15. Matrix Families

Experiments should not use only dense random matrices.

At least the following matrix families should be tested:

15.1 Dense Random

Approximately full-rank and non-sparse.

Purpose:

- control case,
- likely worst case for adaptive preprocessing.

15.2 Block Sparse

Contains entire zero or near-zero regions.

Purpose:

- evaluate Frobenius-norm pruning.

15.3 Low-Rank

Constructed as

$$A = XY^T$$

with rank $r \ll N$.

Purpose:

- evaluate rank detection and factored transmission.

15.4 Mixed Structured

Contains:

- dense blocks,
- sparse blocks,
- low-rank blocks.

Purpose:

- evaluate the full adaptive system under heterogeneous task types.

15.5 Optional Structured Dense

A reproducible structured matrix family may be added as a middle ground between random dense and explicitly low-rank data.

16. Independent Variables

Candidate experimental variables include:

Variable	Candidate Values
Matrix dimension N	100, 500, 1000, 2000, 5000, up to 10000
Worker count P	1, 2, 4, 8
Leaf block size b_{leaf}	128, 256, 512, configurable
ϵ_{zero}	disabled, 10^{-14} , 10^{-12} , 10^{-10} , etc.
ϵ_{rank}	configurable tolerance sweep
r_{max}	configurable rank cap
Matrix family	dense, sparse, low-rank, mixed

The final parameter grid will be reduced if necessary to keep the benchmark campaign tractable.

17. Metrics

The benchmark driver should record:

17.1 Performance

$$T_P$$

$$S_P = \frac{T_1}{T_P}$$

$$E_P = \frac{S_P}{P}$$

as well as:

- total wall-clock time,
- Ingress preprocessing time,
- queue wait time,
- communication time,
- worker compute time,
- aggregation time.

17.2 Communication

- total bytes transmitted,
- total bytes received,
- number of framed messages,
- tasks dispatched,
- tasks pruned,
- dense tasks,
- factored tasks,
- effective compression ratio.

17.3 Accuracy

$$\frac{\|C - \widehat{C}\|_F}{\|C\|_F}$$

plus an exact or tolerance-based numerical comparison against the serial reference result.

17.4 Resource Use

If practical:

- peak memory,
- per-container CPU utilization,
- worker idle time,
- scheduler queue depth.

18. Benchmark Methodology

To improve experimental validity:

1. use fixed random seeds,
2. record hardware and software versions,
3. pre-build and pre-start Docker containers,

4. use persistent worker connections,
5. perform warm-up runs before timed trials,
6. repeat timed experiments,
7. report median and/or mean with variation,
8. do not include container image build time in matrix-multiplication timing,
9. distinguish preprocessing, network, compute, and aggregation time,
10. verify every result against the serial baseline,
11. keep benchmark generation reproducible.

The 10,000-by-10,000 case should be treated as a stress benchmark rather than a routine development input because float64 matrices at that scale consume substantial memory.

19. Implementation Plan

Phase 1 — Correct Serial Reference

- matrix generation/loading,
- validation,
- serial multiplication,
- reference checksum and numerical comparison.

Exit condition: trusted baseline exists.

Phase 2 — Static Docker/TCP Prototype

- Ingress container,
- worker container,
- Aggregator container,
- framed binary TCP protocol,
- dense block payloads,
- correct matrix reconstruction.

Exit condition: distributed result matches serial result.

Phase 3 — Multi-Worker Dynamic Scheduler

- configurable worker count,
- persistent connections,
- central task queue,
- pull-based task assignment,
- timing instrumentation.

Exit condition: correct execution with $P > 1$.

Phase 4 — Quadtree-Compatible Decomposition

- dynamic zero-padding,
- recursive or equivalent leaf-block decomposition,

- block coordinate tracking.

Exit condition: arbitrary supported N values are handled correctly.

Phase 5 — Sparsity Pruning

- Frobenius-norm bound,
- configurable ϵ_{zero} ,
- pruning counters,
- numerical error measurement.

Exit condition: measurable traffic reduction on sparse inputs.

Phase 6 — Benchmark Instrumentation

- bytes,
- tasks,
- messages,
- timings,
- speedup,
- efficiency,
- error,
- reproducible experiment output.

Exit condition: static and pruned systems can be compared experimentally.

Phase 7 — Rank-Adaptive Compression

- truncated/randomized low-rank factorization,
- factored task payloads,
- low-rank worker kernels,
- compression metrics.

Exit condition: measurable low-rank communication experiment.

Phase 8 — Final Experimental Campaign

- dense control,
- sparse,
- low-rank,
- mixed structured matrices,
- worker scaling,
- block-size sweep,
- threshold sweep.

Exit condition: paper-ready tables and plots.

20. Scope Protection

The guaranteed project is complete after Phase 6.

Low-rank transmission is a graduate research enhancement rather than a dependency for basic system correctness.

This prevents the project from failing if:

- SVD preprocessing is too expensive,
- rank detection does not amortize,
- compressed paths require additional debugging.

A negative result is still scientifically useful if the experiment shows that factorization overhead exceeds communication savings for certain matrix classes.

21. Anticipated Areas of Contention

21.1 SVD / Rank-Detection Cost

Risk:

$$T_{\text{factorization}} > T_{\text{communication saved}}.$$

Mitigation:

- truncated/randomized methods,
- minimum block-size threshold,
- rank cap,
- measure preprocessing independently.

21.2 Communication Bottleneck

Risk:

Many small blocks increase

$$q^3$$

task messages and

$$O(N'^3/b)$$

payload traffic.

Mitigation:

- block-size experiments,
- persistent sockets,
- contiguous binary payloads,
- possible future task batching.

21.3 Numerical Approximation

Risk:

Pruning and low-rank compression produce $\hat{C} \neq C$.

Mitigation:

- explicit tolerances,
- relative Frobenius error,
- exact static baseline,
- no claim of exactness when approximation is enabled.

21.4 Ingress Bottleneck

Risk:

The Ingress service performs decomposition, norms, rank analysis, and scheduling.

Mitigation:

- profile each stage,
- defer low-rank analysis until baseline works,
- avoid unnecessary copies,
- optionally parallelize preprocessing only if required.

21.5 Memory Pressure

Risk:

Large dense matrices and temporary buffers can consume multiple gigabytes.

Mitigation:

- stress-test large sizes separately,
- avoid unnecessary block copies,
- use contiguous views/buffers where possible,
- enforce configurable benchmark sizes.

21.6 Workload Imbalance

Risk:

Dense, sparse, and factored tasks have unequal execution times.

Mitigation:

- dynamic pull-based task queue,
- per-task timing,
- worker availability tracking.

21.7 Worker Failure

Risk:

A worker disconnect could leave an output block incomplete.

Mitigation:

- task state tracking,
- socket timeout,
- bounded retry,
- duplicate-result protection.

22. Checkpoint 1 Readiness Target

For Checkpoint 1, the team should be able to explain:

1. why matrix multiplication is suitable for divide-and-conquer parallelization,
2. why Docker/TCP introduces communication costs not present in shared-memory execution,
3. why block size creates a parallelism-versus-communication tradeoff,
4. why sparse and low-rank structure may reduce network traffic,
5. why pruning is approximate rather than exact when $\epsilon_{\text{zero}} > 0$,
6. how the three experimental systems isolate different effects,
7. which matrix families will be tested,
8. which metrics will determine success,
9. which implementation phases come next,
10. which technical risks could alter the final design.

23. Project Success Criteria

The project is successful if it:

- correctly multiplies supported matrices,
- distributes block tasks across multiple Docker workers,
- uses TCP sockets with reliable application-layer framing,
- reconstructs the final result,
- scales worker count,
- records performance and communication metrics,
- demonstrates experimentally when adaptive structure-aware processing helps or hurts,
- quantifies approximation error,
- produces reproducible results suitable for an 8-page graduate IEEE/ACM paper and final presentation/demo.

The central goal is not to claim universal superiority.

The central goal is to identify and explain the conditions under which numerical structure can reduce the communication cost of distributed matrix multiplication over containerized microservices.